

05 — Einfache Datentypen

str, int, float, bool – und was dahinter steckt

HTW Berlin – Wirtschaftsingenieurwesen

SoSe 2026

Agenda

1. Was sind Datentypen? – `type()`
2. `str` – Text und String-Methoden
3. Unter der Haube: Zeichen sind Zahlen (ASCII, Unicode)
4. `int` – ganze Zahlen, in Python grenzenlos
5. `float` – Kommazahlen, IEEE 754, Rundungsfehler
6. `bool` – wahr oder falsch
7. Typumwandlung
8. **Klassiker: Temperatur-Umrechner** – live programmiert

Lernziel: Veggie-Soles-Daten korrekt typisieren – und **verstehen**, wie Python Werte als Bitmuster ablegt und warum `0.1 + 0.2` krumm aussieht.

Heute: nach jedem Konzept *Live-Demo* (ich) und *Sofort ausprobieren* (Sie, Notebook 05).

Wozu Datentypen?

Ohne Typ-Wissen

```
preis = input("Preis: ")      # User tippt 89.95  
print(preis + preis)         # 89.9589.95 (?!)
```

Mit Typ-Wissen

```
preis = float(input("Preis: "))  
print(preis + preis)         # 179.9
```

Gleicher Operator +, völlig anderes Ergebnis – weil der **Datentyp** bestimmt, was eine Operation bedeutet. Heute: die vier Grundtypen und ihre Mechanik.

Was sind Datentypen? - type()

Jeder Wert hat einen **Typ** - type() zeigt ihn:

```
print(type("Eco-Sneaker"))    # <class 'str'>
print(type(42))                # <class 'int'>
print(type(89.95))             # <class 'float'>
print(type(True))              # <class 'bool'>
```

- str Text, int ganze Zahl, float Kommazahl, bool Wahrheitswert
- Der Typ entscheidet, **welche Operationen erlaubt** sind - und **was sie tun**
- Python merkt sich den Typ **am Wert**, nicht am Variablennamen

 **Sofort ausprobieren** - Notebook 05, Kap. 1: type() auf Namen, Zahl und "123" anwenden

Unter der Haube: der Typ bestimmt die Operation

```
print(2 + 3)           # 5  
print("2" + "3")      # 23
```

Ausdruck	Typen	+ bedeutet	Ergebnis
2 + 3	int + int	Addition	5
"2" + "3"	str + str	Verkettung	"23"
"2" + 3	str + int	nicht definiert	TypeError

Rückbezug E4: im Speicher liegen nur **Bitmuster**. Erst der Typ sagt Python, *wie* ein Muster zu lesen ist und *welche* Maschinerie + startet – deshalb prüft Python vor jeder Operation die Typen.

String - Text (str)

```
produkt = "Eco-Sneaker"           # doppelte Quotes
beschr = 'Low-Cut, recycelt'      # einfache Quotes
banner = "=" * 30                 # vervielfältigen
satz = produkt + ": 89.95 EUR"    # verketteten
print(len(produkt))               # 11 Zeichen
```

- Text steht in " oder ' - dreifache """ für mehrere Zeilen
- + verkettet, * wiederholt, len() zählt Zeichen
- "89.95" ist **Text**, keine Zahl – damit rechnet Python nicht

► **Live-Demo** - 01_string_basics.py

String-Methoden

Methoden werden mit `.` am Wert aufgerufen:

```
name = "Eco-Sneaker"
name.upper()           # "ECO-SNEAKER"
name.replace("-", " ") # "Eco Sneaker"
name.startswith("Eco") # True (ein bool!)
"  Anna  ".strip()     # "Anna"
```

- Methoden **verändern den String nicht** – sie liefern einen **neuen** Wert zurück (Strings sind unveränderlich)

► **Live-Demo** – 02_string_methoden.py

🔗 **Sofort ausprobieren** – Notebook 05, Kap. 2: Lieblingsfarbe, Satz mit Anführungszeichen, "999" vs. 999

Unter der Haube: Zeichen sind Zahlen (ASCII)

```
print(ord("A"))    # 65  -- Zeichen -> Nummer  
print(chr(66))     # B   -- Nummer  -> Zeichen
```

Ausschnitt aus der **ASCII-Tabelle** (128 Zeichen, seit 1963):

Zeichen	Code	Zeichen	Code
"0"	48	"A"	65
"9"	57	"a"	97

Rückbezug E4: 01000001 ist als Zahl die **65**, als Zeichen das **"A"** – derselbe Speicherinhalt, andere Lesebrille. Ein String ist intern eine **Folge solcher Zeichen-Nummern**.

Unter der Haube: Unicode & UTF-8

ASCII kennt nur 128 Zeichen – keine Umlaute, keine Emojis. **Unicode** gibt *jedem* Zeichen der Welt eine Nummer (*Codepoint*):

```
print(ord("ä"))      # 228
print(ord("€"))      # 8364
print(chr(127793))   # Setzling-Emoji
```

Zeichen	Codepoint	Platz in UTF-8
"A"	65	1 Byte
"ä"	228	2 Bytes
"€"	8364	3 Bytes

- **UTF-8** = Speicherformat: häufige Zeichen kurz, seltene länger
- Wichtig ab Notebook 12 (Dateien) – dort heißt das encoding

Integer - ganze Zahlen (int)

```
bestand = 42
verkauft = 7
print(bestand - verkauft)    # 35
print(17 / 5)                # 3.4  Division -> immer float
print(17 // 5)               # 3    ganzzahlige Division
print(17 % 5)                # 2    Rest ("modulo")
```

- Ganze Zahlen: positiv, negativ, null – für Stückzahlen, IDs
- Operatoren: + - * / // % ** (Potenz)
- / liefert **immer** float – auch 10 / 2 ergibt 5.0

 **Sofort ausprobieren** – Notebook 05, Kap. 3: Bestandsrechnung, // und % testen

Unter der Haube: int ist in Python grenzenlos

```
print(2 ** 100)  
# 1267650600228229401496703205376
```

- In C/Java hat int **fix 32/64 Bit** – wird die Zahl zu groß, läuft sie über
- Python-int wächst **beliebig**: mehr Ziffern → Python hängt automatisch weitere Speicherblöcke an → Ergebnis bleibt **exakt**

Division mit Rest, formal: für $b \neq 0$ gilt stets

$$a = b \cdot \lfloor a/b \rfloor + (a \bmod b)$$

Probe mit $a = 17, b = 5$: $17 = 5 \cdot 3 + 2$. // und % sind **ein Paar** – zusammen rekonstruieren sie a verlustfrei.

Float - Kommazahlen (float)

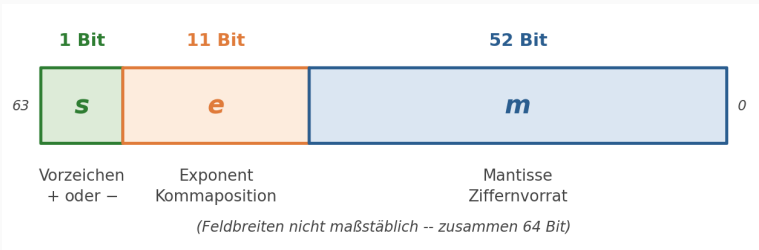
```
preis = 89.95           # Punkt, kein Komma!  
mwst = 0.19  
brutto = preis * (1 + mwst)  
print(brutto)           # 107.0405  
print(f"{brutto:.2f} EUR") # 107.04 EUR
```

- Für Preise, Maße, Anteile – wissenschaftlich: $1.5e3$ ist 1500.0
- Anzeige runden: f-String `:.2f` – Wert runden: `round(brutto, 2)`
- Manche Rechnungen werden **krumm** ($0.1 + 0.2$) – gleich sehen wir, warum

► **Live-Demo** – 03_zahlen_rechnen.py

Unter der Haube: float ist IEEE 754

Ein float hat **genau 64 Bit** – aufgeteilt in drei Felder:



Der Wert entsteht als $(-1)^s \cdot m \cdot 2^e$ – Vorzeichen s , Mantisse m (Ziffernvorrat), Exponent e (Kommaposition).

64 Bit $\Rightarrow$ nur **endlich viele** Zahlen sind exakt darstellbar – alles dazwischen wird **gerundet** (wie $1/3$ dezimal: $0.333\dots$ wird irgendwo abgeschnitten).

Unter der Haube: warum $0.1 + 0.2$ krumm ist

```
print(0.1 + 0.2)          # 0.30000000000000004
print(0.1 + 0.2 == 0.3)   # False (!)
```

- 0.1 ist **binär periodisch**: $0.000110011001100\dots$ – wie $1/3$ dezimal
- Nach 52 Mantissen-Bits schneidet IEEE 754 ab $\rightarrow$ winziger Fehler, sichtbar nach dem Rechnen

Praxisregeln für Preise – beide Ausdrücke sind True:

```
round(0.1 + 0.2, 2) == 0.3      # runden
abs((0.1 + 0.2) - 0.3) < 0.001  # Toleranz
```

Nie Floats mit `==` vergleichen – immer runden oder mit Toleranz arbeiten.

Boolean - wahr oder falsch (bool)

```
auf_lager = True           # oder False
print(89.95 > 100)         # False
print("Eco" in "Eco-Sneaker") # True
print(89.95 > 50 and auf_lager) # True
```

- Ergebnis aller **Vergleiche** (== != < > <= >=) und von in auf Strings
- and/or/not verknüpfen Bool-Werte – Grundidee heute, **Vertiefung in Notebook 08**

► **Live-Demo** – 04_boolean_bedingungen.py

✎ **Sofort ausprobieren** – Notebook 05, Kap. 5: Vergleiche bauen, bool()-Fallen testen

Unter der Haube: bool ist eine 1-Bit-Zahl

```
print(int(True), int(False))    # 1 0
print(True + True)              # 2 -- kurios, aber echt
```

Ausdruck	die ALU rechnet	Ergebnis-Bit
89.95 > 100	Vergleich	0 → False
89.95 < 100	Vergleich	1 → True

- Ein Vergleich ist eine **Rechenoperation** der ALU (E4!) mit Ergebnis 0 oder 1
- Python verpackt dieses Bit als False/True – intern ist bool ein Spezialfall von int

Typumwandlung - int(), float(), str()

Python wandelt **nie automatisch** zwischen str und Zahl:

```
anzahl = int(input("Anzahl: "))    # str -> int
preis = float("89.95")             # str -> float
text = "ab " + str(89.95)          # float -> str
```

Funktion	wandelt nach
int(x) / float(x)	Zahl
str(x)	Text
bool(x)	True/False

► **Live-Demo** - 05_typumwandlung.py

Unter der Haube: was bei der Umwandlung passiert

Umwandlung liest den alten Wert und **baut einen neuen Wert** – das Original bleibt unverändert. Und sie kann scheitern:

Aufruf	Ergebnis	Warum
<code>int("42")</code>	42	Ziffern gelesen, Zahl gebaut
<code>int("3.9")</code>	ValueError	kein int-Format
<code>int(3.9)</code>	3	schneidet ab – rundet nicht!
<code>bool("False")</code>	True	nicht-leerer String

Faustregel: `input()` liefert **immer** `str` – vor dem Rechnen `int()` oder `float()` drumherum. Genau das braucht gleich unser Klassiker.

Klassiker: Temperatur-Umrechner - die Aufgabe

Das US-Lager braucht die Soll-Lagertemperatur der **Bambus-Boots** in Fahrenheit – bei uns steht sie in Celsius:

- Celsius **einlesen** – Achtung: `input()` liefert `str`!
- Umrechnen: $F = C \cdot 9/5 + 32$
- **Formatiert** ausgeben: 2 Nachkommastellen (f-String)
- Rückprobe in Gegenrichtung: $C = (F - 32) \cdot 5/9$

So soll es laufen

Temperatur in Grad C: 21.5

21.50 Grad C = 70.70 Grad F

Rueckprobe: 70.70 Grad F = 21.50 Grad C

Klassiker: Temperatur-Umrechner - Live-Coding

► **Live-Coding** ab **leerem Editor** - Referenz: 90_klassiker_temperatur.py

Teilziele:

1. Eingabe einlesen: `float(input(...))` - `input()` liefert String!
2. Formel als Ausdruck: `fahrenheit = celsius * 9 / 5 + 32`
3. Ausgabe mit f-String und `:.2f`
4. Rückrichtung als Rückprobe: `(fahrenheit - 32) * 5 / 9`

✎ **Zum Selberlösen** - Handout "*Klassiker 05: Temperatur-Umrechner*" (Klassiker/05_Klassiker_Temperatur.pdf). Vertiefung: Notebook 05, Trainingsmaterial.

Cheat Card

Aufgabe	Syntax
Typ herausfinden	<code>type(x)</code>
Text verketteten / wiederholen	<code>"a" + "b", "-" * 30</code>
String-Methode aufrufen	<code>name.upper(), name.strip()</code>
Zeichen ↔ Nummer	<code>ord("A"), chr(65)</code>
Ganzzahl-Division / Rest	<code>17 // 5, 17 % 5</code>
Umwandeln	<code>int(x), float(x), str(x)</code>
Float ausgeben / vergleichen	<code>f"{x:.2f}", abs(a-b) < 0.001</code>
Klassiker heute	<code>float(input()) + Formel + :.2f</code>

Faustregel: `input()` liefert immer str. Preise sind float – ausgeben mit `:.2f`, vergleichen nie mit `==`.

Ausblick: Notebook 06 – komplexe Datentypen

Unser Umrechner schafft pro Lauf **eine** Temperatur – und Veggie Soles hat mehrere Lager:

```
temp_berlin = 18.5  
temp_hamburg = 19.0  
temp_austin = 21.5      # ...und bei 50 Lagern?
```

Einfache Typen halten **einen** Wert. Nächste Woche: **Listen, Tupel, Sets und Dictionaries** – Behälter, die viele Werte in *einer* Variablen bündeln: der ganze Warenkorb, der komplette Produktkatalog.

Heute geübt

- ✓ `type()` – der Typ bestimmt, was eine Operation tut
- ✓ `str` + Methoden; Zeichen sind Zahlen (`ord/chr`, ASCII, Unicode)
- ✓ `int` – in Python beliebig groß; `/` vs. `//` vs. `%`
- ✓ `float` – IEEE 754: 64 Bit, deshalb ist `0.1 + 0.2` krumm
- ✓ `bool` – Ergebnis von Vergleichen, intern eine 1-Bit-Zahl
- ✓ Typumwandlung – `input()` liefert immer `str`
- ✓ **Klassiker Temperatur-Umrechner** live gesehen

Ihre Aufgaben bis zur nächsten Woche:

- Klassiker **Temperatur-Umrechner** eigenständig lösen (Handout; Bonus: Kelvin)
- “Sofort ausprobieren”-Aufgaben im Notebook 05
- Reflexionsfrage: warum zeigt `print(0.1 + 0.2)` etwas Krummes?